DAYANANDA SAGAR UNIVERSITY

Devarakaggalahalli, Harohalli

Kanakapura Road, Bengaluru South Dt - 562112, Karnataka, India

Bachelor of Technology

in

COMPUTER SCIENCE AND ENGINEERING

(CYBER SECURITY)

Mini Project Report On

SMART INVENTORY AND BILLING MANAGEMENT SYSTEM

Batch: 2

By

KAVEEN RAAJ M	ENG24CY0122
GAVISIDDESH KATTI	ENG24CY0024
S. NIVETHA	ENG24CY0191
ABINAYA P	ENG24CY0075

Under the supervision of

Dr. Indushree M

Assistant Professor

DEPARTMENT OF CSE (CYBER SECURITY)

SCHOOL OF ENGINEERING

DAYANANDA SAGAR UNIVERSITY

(2025-2026)

DAYANANDA SAGAR UNIVERSITY

Department of Computer Science & Engineering (Cyber Security)

Devarakaggalahalli, Harohalli, Kanakapura Road, Bengaluru South Dt - 562112

Karnataka, India

SCHOOL OF ENGINEERING

CERTIFICATE

This is to certify that the Major Project Stage-I work titled "**SMART INVENTORY AND BILLING MANAGEMENT SYSTEM**" is carried out by Kaveen Raaj M (ENG24CY0122), Gavisiddesh Katti (ENG24CY0024), S. Nivetha (ENG24CY0191), and Abinaya P (ENG24CY0075), bonafide students of the Fourth semester Bachelor of Technology in Computer Science and Engineering (Cyber Security) at the School of Engineering, Dayananda Sagar University, Bangalore, in partial fulfilment for the award of the degree of Bachelor of Technology in Computer Science and Engineering (Cyber Security), during the academic year 2025-2026.

Dr. Indushree M
Assistant Professor
Dept. of Cyber Security
School of Engineering
Dayananda Sagar University

Date: 30/04/2026

Dr. Dilip Kumar Saini
Dept of Cyber Security
School of Engineering
Dayananda Sagar University

Date: 30/04/2026

DECLARATION

We, Kaveen Raaj M (ENG24CY0122), Gavisiddesh Katti (ENG24CY0024), S. Nivetha (ENG24CY0191), and Abinaya P (ENG24CY0075), are students of the Fourth semester B.Tech in Computer Science and Engineering (Cyber Security), at the School of Engineering, Dayananda Sagar University. We hereby declare that the Mini Project titled "SMART INVENTORY AND BILLING MANAGEMENT SYSTEM" has been carried out by us and submitted in partial fulfilment for the award of the degree of Bachelor of Technology in Computer Science and Engineering (Cyber Security) during the academic year 2025-2026.

Name	USN	Signature
Kaveen Raaj M	ENG24CY0122	
Gavisiddesh Katti	ENG24CY0024	
S. Nivetha	ENG24CY0191	
Abinaya P	ENG24CY0075	

Place: Bangalore

Date: 30/04/2026

ABSTRACT

The Smart Inventory and Billing Management System is a web-based database management application designed to simplify inventory control, customer billing, invoice generation, and sales monitoring for small and medium-scale retail environments. Traditional inventory and billing practices often rely on manual ledgers, spreadsheets, or isolated desktop tools, which can create billing errors, delayed stock updates, and poor visibility into daily business performance.

The proposed system integrates inventory management and point-of-sale billing into a single platform. It uses a React-based frontend for interactive user operations, a Node.js and Express.js backend for business logic, and a relational database managed through Sequelize ORM for structured data storage. The system supports role-based access control, secure password handling, product and customer management, invoice creation, automated stock deduction, low-stock visibility, and dashboard-level reporting.

The project demonstrates the practical application of DBMS concepts such as relational schema design, primary and foreign key constraints, CRUD operations, transaction safety, role-based data access, and structured query processing. By linking billing directly with inventory records, the system reduces manual reconciliation work and improves operational consistency. The final implementation provides a maintainable foundation that can be extended with barcode scanning, payment gateway integration, advanced analytics, and cloud deployment in future versions.

ACKNOWLEDGEMENT

It is a great pleasure for us to acknowledge the assistance and support of many individuals who have contributed to the successful completion of this mini project work.

We express our sincere gratitude to the School of Engineering and Technology, Dayananda Sagar University, for providing us with the opportunity and academic environment required to complete this project.

We would like to thank Dr. Udaya Kumar Reddy K R, Dean, School of Engineering, Dayananda Sagar University, for his constant encouragement and support.

We sincerely thank Dr. Dilip Kumar Saini, Chairman, Department of Computer Science and Engineering (Cyber Security), Dayananda Sagar University, for his academic guidance and support.

We are deeply grateful to our project guide, Dr. Indushree M, Assistant Professor, Department of Computer Science and Engineering (Cyber Security), Dayananda Sagar University, for her valuable time, guidance, and support throughout the project work.

We also thank our family, friends, and everyone who directly or indirectly supported us during the completion of this project.

LIST OF FIGURES

LIST OF TABLES

TABLE OF CONTENTS

CHAPTER 1

INTRODUCTION

1.1 OVERVIEW

In the contemporary retail environment, businesses generate and process a large volume of transactional data through purchases, sales, stock updates, invoices, and customer records. When these activities are handled manually or through disconnected tools, the result is often delayed billing, inaccurate stock records, repeated data entry, and poor visibility into business performance.

The Smart Inventory and Billing Management System, referred to as SmartBiz IMS, is developed as a full-stack web application to address these operational limitations. The system connects inventory control, customer management, invoice generation, role-based access, and dashboard reporting into a single database-backed platform. It is designed to support store owners, administrators, managers, cashiers, and viewers according to their assigned level of access.

The system uses a client-server architecture. The frontend is implemented using React.js, while the backend is implemented using Node.js and Express.js. The data layer uses a relational database accessed through Sequelize ORM. This structure allows the system to separate presentation, business logic, and persistent storage, improving maintainability and reducing the possibility of inconsistent data handling.

1.2 OBJECTIVES

1. To design and develop a web-based inventory and billing management system for small and medium-scale retail operations.
2. To automate invoice generation, subtotal calculation, discount application, tax calculation, and stock deduction.
3. To maintain accurate product, customer, purchase, invoice, and user role data using a relational database structure.

4. To implement role-based access control so that administrative and billing functions are available only to authorized users.
5. To provide a clean dashboard that supports basic business visibility through revenue, invoice, low-stock, and customer statistics.
6. To create a maintainable full-stack application that can be extended with future modules such as barcode scanning, cloud deployment, and payment gateway integration.

1.3 SCOPE

The scope of this project is limited to browser-based retail administration through a centralized backend server. The system includes modules for inventory tracking, customer records, purchase records, user roles, dashboard reporting, and point-of-sale billing. The current version focuses on software workflows and database correctness. It does not include direct integration with physical billing printers, barcode scanners, biometric payment systems, or third-party payment gateways.

CHAPTER 2

PROBLEM DEFINITION

Small and medium-scale retail businesses often operate with fragmented workflows. Billing may be performed in one tool, stock records may be maintained in spreadsheets, and sales reports may be prepared manually at the end of the day. This separation creates avoidable operational risk and consumes time that should be used for business monitoring and customer service.

2.1 EXISTING PROBLEMS

- Data fragmentation: Billing, inventory, purchase, and customer records are often stored separately, which makes reconciliation difficult.
- Billing errors: Manual subtotal, tax, and discount calculations can lead to incorrect invoices and financial mismatch.
- Delayed stock updates: Product quantity is not always updated immediately after a sale, causing wrong availability information.
- Weak accountability: Shared systems and unrestricted access make it difficult to track which user created, edited, or deleted a record.
- Limited reporting: Manual reporting does not provide immediate visibility into revenue, pending invoices, low-stock items, or customer activity.
- Poor scalability: Spreadsheet-based and desktop-only systems become difficult to manage when product volume, customer count, or billing frequency increases.

2.2 PROBLEM STATEMENT

There is a need for an integrated inventory and billing management system that can store business data in a structured relational database, automate billing and stock deduction, protect administrative functions through role-based access control, and provide basic analytical visibility through a web dashboard. The system must reduce manual work, improve data consistency, and support reliable day-to-day retail operations.

CHAPTER 3

LITERATURE SURVEY

The SmartBiz IMS project is based on established ideas from inventory management, web application architecture, relational database design, and secure backend development. This chapter reviews the technical background required to justify the selected design.

3.1 EVOLUTION OF INVENTORY MANAGEMENT SYSTEMS

Traditional inventory management relied on physical ledgers and manual stock cards. Later systems shifted to spreadsheet-based records and desktop database applications. These methods improved storage but still created limitations in multi-user access, live stock synchronization, and centralized reporting. Modern systems increasingly use web-based interfaces connected to centralized databases so that inventory and billing operations can be updated immediately across users.

3.2 DIGITAL TRANSFORMATION IN SMES

Small and medium enterprises require systems that are affordable, easy to maintain, and capable of handling core business operations without heavy infrastructure. A browser-based application is suitable for this environment because it can run on ordinary client machines while maintaining business logic and database control on a central server.

3.3 FULL-STACK ARCHITECTURAL PARADIGMS

Full-stack web applications commonly separate the user interface, server-side logic, and database storage. React.js supports a component-based frontend, while Node.js with Express.js provides a lightweight REST API backend. This approach allows the point-of-sale interface, dashboard, and administrative modules to interact with the same backend services without duplicating logic.

3.4 RELATIONAL INTEGRITY AND PERFORMANCE

Billing systems require reliable relationships between products, customers, invoices, invoice items, users, and stock records. A relational database is well-suited for this requirement because it supports primary keys, foreign keys, constraints, and transactions. Sequelize ORM provides an abstraction layer that maps backend models to relational tables while still preserving structured database relationships.

Table 3.1: Comparison of existing and proposed systems

Feature Category	Traditional System	SmartBiz IMS
Architecture	Standalone desktop or spreadsheet-based workflow	Web-based client-server architecture with REST API
Data Storage	Local files or manually maintained records	Centralized relational database
Inventory Update	Manual or delayed stock correction	Stock deduction linked with invoice creation
Authentication	Shared or weak user control	Role-based access control with protected routes
Reporting	Manual end-of-day calculation	Dashboard-level revenue, invoice, stock, and customer visibility

CHAPTER 4

PROJECT DESCRIPTION

The SmartBiz IMS project implements a structured 3-tier system with a React client, an Express backend, and a relational database layer. The design focuses on clean separation of responsibilities so that user interface logic, business validation, and database operations remain maintainable.

4.1 PROPOSED DESIGN

4.1.1 Presentation Tier

The presentation tier is implemented using React.js. It provides the dashboard, inventory interface, purchase module, billing screen, customer module, and reports interface. React Router is used to manage navigation between protected pages. The billing screen maintains the active cart, selected products, quantities, discounts, tax values, and total amount before sending the final invoice request to the backend.

4.1.2 Application Service Tier

The application service tier is implemented using Node.js and Express.js. It exposes REST API endpoints for login, inventory, customers, purchases, invoices, reports, and users. Middleware components are used for authentication, role checks, input validation, error handling, and rate control. This layer performs the main business logic, including authorization decisions and server-side invoice processing.

4.1.3 Data Tier

The data tier uses a relational database accessed through Sequelize ORM. Tables are modeled for products, users, customers, purchases, invoices, invoice items, and related business records. The ORM model definitions help enforce relationships and simplify query operations while the relational database maintains persistent storage.

4.2 USER ROLES AND PERMISSIONS

The system uses role-based access control to prevent unauthorized modification of sensitive data. The permission model assigns different operational capabilities to Super Admin, Admin, Manager, Cashier, and Viewer roles.

Table 4.1: User role permission matrix

User Role	Dashboard & Analytics	Checkout POS	Edit Inventory	Delete Invoice History	System Configurations
Super Admin	Yes	Yes	Yes	Yes	Yes
Admin	Yes	Yes	Yes	No	Yes
Manager	Yes	Yes	Limited	No	No
Cashier	No	Yes	No	No	No
Viewer	Yes	No	No	No	No

4.3 ASSUMPTIONS AND DEPENDENCIES

4.3.1 Assumptions

1. Client machines are connected to the same local server or deployed backend server.
2. Initial product, user, and tax configuration is entered by an authorized administrator.
3. The system is intended for small and medium-scale retail workflows, not large enterprise ERP replacement.
4. Each product has a unique identifier and a maintained stock quantity.
5. Invoice creation is treated as the authority for stock deduction.

4.3.2 Dependencies

1. Node.js runtime environment for executing the backend server.
2. npm package manager for dependency installation and build operations.
3. React.js and Vite for frontend development.
4. Express.js for backend API routing.
5. Sequelize ORM for database model mapping.
6. SQLite for development or MySQL for production deployment.
7. Modern web browser supporting JavaScript, CSS Grid, and Flexbox.

CHAPTER 5

REQUIREMENTS

5.1 FUNCTIONAL REQUIREMENTS

Functional requirements define the operations that the system must perform during regular use.

5.1.1 Admin and Manager Module

1. The system shall allow authorized users to add, update, view, and manage product records.
2. The system shall allow authorized users to manage customer records.
3. The system shall allow authorized users to view dashboard statistics and reports.
4. The system shall generate low-stock visibility based on configured product quantities.
5. The system shall restrict inventory deletion when related invoices or historical records are linked.

5.1.2 Billing Module

1. The system shall allow cashiers to search products and add items to a billing cart.
2. The system shall calculate subtotal, discount, tax, and final payable amount.
3. The system shall generate an invoice after successful checkout.
4. The system shall deduct inventory quantity only after invoice creation is validated.
5. The system shall support invoice export or print-ready invoice output.

5.2 NON-FUNCTIONAL REQUIREMENTS

5.2.1 Security

Passwords should be stored using secure hashing. Protected API routes should require a valid authenticated session or bearer token.

5.2.2 Performance

Dashboard and billing operations should return responses within acceptable time under normal local or departmental deployment conditions.

5.2.3 Usability

The user interface should be clear enough for non-technical retail staff to operate without complex training.

5.2.4 Maintainability

Frontend components, backend controllers, routes, middleware, and database models should remain separated for easier debugging and future enhancement.

5.2.5 Reliability

Billing and stock deduction should be handled together to avoid inconsistent inventory states.

5.3 SOFTWARE AND HARDWARE REQUIREMENTS

Table 5.1: Software and hardware requirements

Category	Requirement	Specification
Software	Operating System	Windows 10 or above, Linux Ubuntu, or macOS
Software	Development Environment	Visual Studio Code or IntelliJ IDEA
Software	Frontend Framework	React.js with Vite
Software	Backend Framework	Node.js and Express.js
Software	Styling Engine	Tailwind CSS or custom responsive CSS
Software	Database / ORM	SQLite or MySQL with Sequelize ORM
Software	API Testing Tool	Postman, Insomnia, or cURL
Software	Package Manager	npm
Hardware	Processor	Dual-core 2.0 GHz or higher
Hardware	RAM	4 GB minimum; 8 GB recommended
Hardware	Storage	At least 500 MB free space for application files and database
Hardware	Network	Localhost, LAN, or Internet connection depending on deployment

CHAPTER 6

IMPLEMENTATION APPROACH

6.1 IMPLEMENTATION STRATEGY

The implementation strategy was divided into three major phases: database modeling, API construction, and frontend integration. This phased approach reduces implementation risk and keeps the application easier to test.

6.1.1 Data Modeling and Migrations

The database schema was designed around core retail entities such as users, roles, products, customers, purchases, invoices, and invoice items. Product and invoice relationships were modeled to ensure that a billed item remains linked with the product record used during checkout. Restrictive delete rules are preferred for financial and historical data because invoices should remain auditable after creation.

6.1.2 API Construction and Authorization Flow

Express routers were mapped according to functional modules. Routes were grouped for authentication, inventory, purchases, customers, billing, and reporting. Authentication middleware validates requests before allowing protected actions. Role-check middleware ensures that users can perform only the operations assigned to their role.

6.1.3 React Client Integration

The frontend was organized into reusable components such as sidebar navigation, cards, forms, product tiles, cart panels, and dashboard widgets. Axios or equivalent HTTP clients are used to send API requests. Billing calculations are displayed immediately on the client side for usability, while the backend remains responsible for final validation before invoice creation.

6.1.4 Billing Logic

The billing logic follows a simple and auditable sequence: select product, enter quantity, compute subtotal, apply discount, compute tax, generate invoice, and update stock. This sequence prevents manual mismatch between billing and inventory records.

Subtotal = Quantity x Unit Price

Tax Amount = (Subtotal - Discount) x Tax Rate / 100

Net Payable = Subtotal - Discount + Tax Amount

CHAPTER 7

EXPERIMENTATION

Testing was carried out to validate authentication, inventory protection, billing correctness, and invoice rendering. The focus was on checking practical edge cases that commonly affect inventory and POS systems.

7.1 TEST STRATEGY

The system was tested using a local development environment. API-level tests were performed using request tools such as Postman or browser-based flows. Interface tests were performed through the dashboard and billing screens.

Table 7.1: Software testing and edge-case validations

Test Case ID	Test Scenario	Input Trigger	Expected Outcome	Status
TC-001	Unauthorized access	Dashboard API request without valid token	Request blocked with unauthorized response	Pass
TC-002	Negative stock checkout	Bill quantity greater than available stock	Checkout rejected and stock unchanged	Pass
TC-003	Role restriction	Cashier attempts inventory edit	Access denied by role-check logic	Pass
TC-004	Invoice generation	Create invoice from valid cart	Invoice created and payable amount displayed	Pass
TC-005	Stock deduction	Create invoice for selected product quantity	Available quantity reduced correctly	Pass

7.2 OBSERVATIONS

- Protected routes rejected unauthorized access attempts.
- Checkout validation prevented stock quantities from going below zero.
- Role-based access control separated cashier functions from inventory administration.
- Invoice creation and stock deduction operated as a linked workflow.
- The dashboard provided quick visibility into revenue, pending invoices, low-stock items, and customer count.

CHAPTER 8

RESULT AND DISCUSSION

8.1 SYSTEM OUTPUT

The completed system provides a functional dashboard, product and inventory module, customer records, billing interface, invoice creation flow, role-based access, and reporting view. The dashboard presents business status indicators such as total revenue, pending invoices, low-stock items, and customer count.

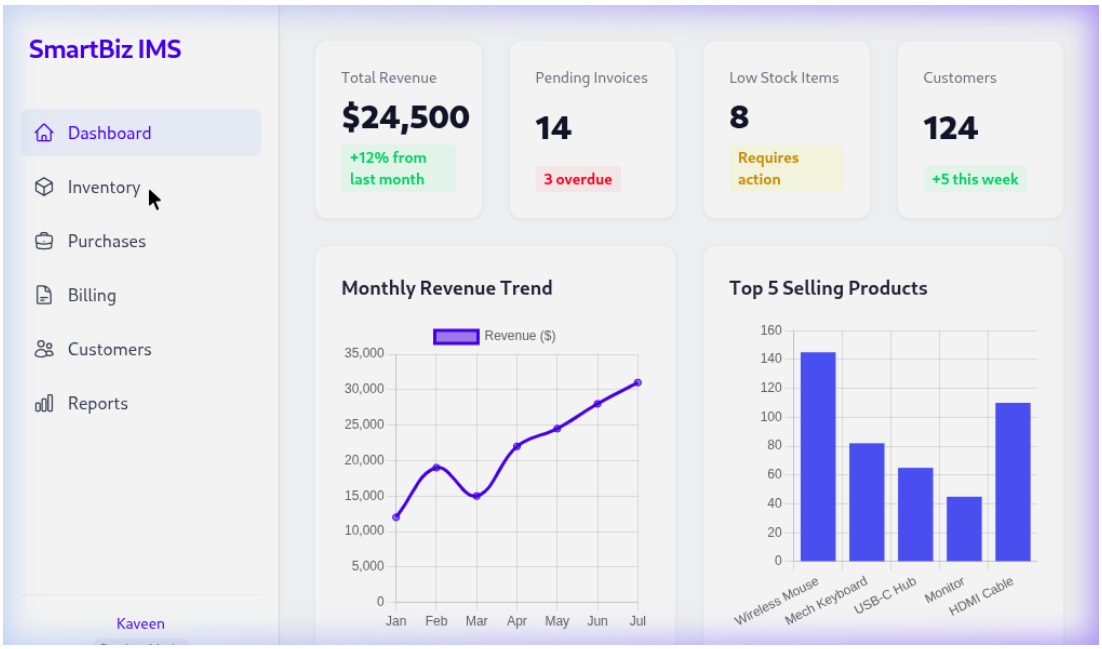

Fig. 8.1 Graphical dashboard

Figure 8.1 shows the dashboard interface of SmartBiz IMS. It displays summary cards and visual charts that help the user monitor sales and stock status without manually checking multiple records.

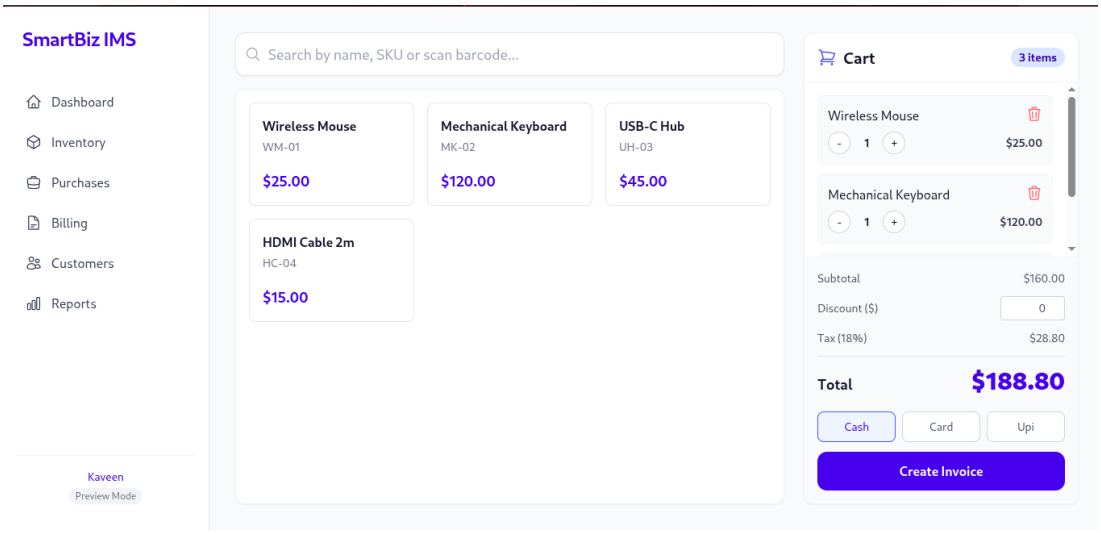

Fig. 8.2 Billing invoice interface

Figure 8.2 shows the billing interface. Products can be searched and added to the cart, while subtotal, discount, tax, and total amount are displayed before invoice creation.

8.2 ACHIEVEMENT OF OBJECTIVES

1. The system integrates inventory management and billing into one platform.
2. Billing calculations are automated and displayed in the interface.
3. Stock quantity is linked to invoice creation, reducing manual correction work.
4. Role-based access separates administrative actions from cashier actions.
5. The dashboard gives a practical overview of business activity.

8.3 LIMITATIONS

1. The current system does not include physical barcode scanner integration.
2. The system does not include direct payment gateway integration in the present version.
3. Advanced accounting features such as GST return filing, profit-loss statements, and supplier credit tracking are outside the current scope.
4. Cloud deployment, backup automation, and high-availability configuration are not implemented in the basic version.

8.4 FUTURE ENHANCEMENTS

1. Barcode scanning support for faster product entry.
2. Payment gateway integration for digital payments.
3. Cloud deployment with automatic database backup.
4. Advanced analytics using monthly sales trends, category-level performance, and demand forecasting.
5. Thermal printer support for direct invoice printing.
6. Audit log dashboard for tracking user-level changes.

REFERENCES

1. E. Gamma, R. Helm, R. Johnson, and J. Vlissides, Design Patterns: Elements of Reusable Object-Oriented Software. Reading, MA: Addison-Wesley, 1994.
2. S. Newman, Building Microservices: Designing Fine-Grained Systems. Sebastopol, CA: O'Reilly Media, 2015.
3. Express.js Documentation, "Express - Fast, unopinionated, minimalist web framework for Node.js." [Online]. Available: https://expressjs.com.
4. React Documentation, "React - The library for web and native user interfaces." [Online]. Available: https://react.dev.
5. Sequelize Documentation, "Sequelize ORM Documentation." [Online]. Available: https://sequelize.org.
6. Node.js Documentation, "Node.js JavaScript runtime." [Online]. Available: https://nodejs.org/en/docs.
7. MDN Web Docs, "JavaScript, HTML, and CSS Documentation." [Online]. Available: https://developer.mozilla.org.